Food Waste Reduction cum Management System: An AI-Driven Approach to Demand Forecasting, Dynamic Pricing, and Equitable Surplus Redistribution in Institutional Cafeterias

Kamalesh P*, Mithul Aaditya A*, Jayaprakash A*, Karthika Lakshmi V*, M. Bhavani†
*Department of Computer Science and Engineering
Rajalakshmi Engineering College, Chennai, India
†Assistant Professor and Project Guide
Department of Computer Science and Engineering
Rajalakshmi Engineering College, Chennai, India

***Abstract*—Cafeterias attached to corporate campuses, colleges, and schools still largely run on guesswork: one menu, one price list, and preparation quantities set by whoever is on duty that morning, regardless of whether the crowd that day is a price-sensitive school canteen or a premium corporate one. That mismatch shows up as food thrown away at close of service and as surplus that, even when it exists, usually goes to whichever NGO happened to be called first. This paper describes a working system built to address that gap, developed as part of an undergraduate project aligned with UN Sustainable Development Goal 12 (Responsible Consumption and Production). The system couples seven cooperating decision engines: an XGBoost model that forecasts daily demand per dish, a closed-form pricing rule that adjusts price within a cost-recovery floor and a segment-specific ceiling, a Random Forest model that predicts how much of a dish will be left over, a transparent scoring function that recommends tomorrow's menu, a Gradient Boosting classifier that targets discount offers at customers likely to convert, and a constraint-based optimizer that splits surplus food fairly across NGO partners. A supervisor process runs all seven in sequence once a day, logging every decision it makes. Because no public dataset records the combination of unit cost, quantity prepared, and institutional segment this system needs, we built a documented synthetic data generator and evaluated the three trained models against a strict, time-ordered holdout rather than a random split. On that generated dataset the forecasting model reaches an R^2 of 0.875, a 45.0% error reduction over a naive weekly-average baseline; the waste model reaches an R^2 of 0.839; and the conversion model achieves an AUC of 0.658 with 2.03× lift over blanket discounting. The allocation engine keeps every NGO partner above its guaranteed 20% share while covering 99.4% of available surplus, and the pricing floor was never once breached across all recorded runs. We report these results candidly, alongside the project's honest limitations, and describe a specific fairness/capacity interaction in the NGO allocator that we found and fixed during development.**

***Index Terms*—food waste management, demand forecasting, dynamic pricing, surplus food redistribution, XGBoost, random forest, constraint programming, SDG 12, institutional food service**

I. Introduction

Walk into most corporate, college, or school cafeterias and the planning process looks roughly the same: a kitchen manager decides how much to cook based on memory and habit, a printed price list stays fixed for weeks or months regardless of demand, and whatever food is left at the end of the day is either thrown out or handed to whichever charity happens to be reachable that evening. This works reasonably well until it doesn't – a rainy Tuesday, an exam week, a corporate holiday – at which point the mismatch between what was cooked and what was actually needed becomes visible as either a stockout or a bin full of wasted food. The problem is compounded by the fact that these three kinds of institutions are not the same market. A corporate cafeteria serves people who are not especially price-sensitive and expect a wider, better menu; a college serves a population that chases whatever dish is trending that month; a school serves a population where every rupee of price increase visibly changes how many students buy lunch. Running all three under one undifferentiated pricing sheet leaves money on the table in the first case and suppresses demand in the third.

This project set out to replace that guesswork with a working, testable system rather than a purely conceptual one. Before writing any code, our team completed a literature survey covering 21 sources across four separate research threads – institutional demand forecasting, personalized food recommendation, AI-driven waste-prevention, and equitable multi-recipient surplus donation – and found that while each of these problems has been studied on its own, none of the reviewed work combined them into a single system for institutional food service, and none addressed segment-differentiated pricing or an enforced cost-recovery floor at all [22]. That survey proposed an architecture to close this gap but did not build or measure it. What follows in this paper is the system we actually built from that proposal, what changed once we started implementing it, and what it measures when run against

a documented dataset.

Concretely, this paper's contribution is fourfold:

1) A synthetic data generator, with every generative assumption written down rather than left implicit, that produces a year of segment-differentiated daily sales across three institution types plus 45 days of individual customer orders – because no existing public dataset carries the unit-cost, preparation-quantity, and institutional-segment fields these engines need.
2) Seven decision engines coordinated as a daily pipeline of specialized agents (context gathering, forecasting, waste prediction, pricing, menu recommendation, offer targeting, and NGO allocation), each agent's failure isolated from the rest so a single bad day does not take down the whole run.
3) A deliberately mixed toolkit rather than one model applied everywhere – three trained models, one exact constraint solver, one rule-based scorer, and one closed-form pricing formula – chosen engine by engine based on whether the decision actually needs statistical learning, hard guarantees, or human-readable justification.
4) Honest, held-out evaluation of every trained model against naive baselines, including one fairness/capacity bug in the NGO allocator that we found mid-project, fixed, and can show the before-and-after numbers for.

Section II summarizes the literature this system builds on and restates the specific gap it targets. Section III lays out the overall architecture and the technology it runs on. Section IV explains how the dataset was generated. Section V walks through each of the seven engines in turn. Section VI reports what we measured. Section VII discusses the design choices we would defend and the limitations we would not hide from an examiner. Section VIII concludes.

II. Related Work

A. Forecasting Demand in Institutional Food Service

Several groups have applied machine learning to the specific problem of predicting how much food an institutional kitchen will need. Aydın et al. evaluated a range of ML techniques on a real institutional cafeteria's demand history, establishing a practical baseline for the problem [1], and a separate study of a university refectory reached a similar conclusion using its own demand records [2]. Agarwal et al. found that blending classical statistical methods with machine learning outperformed either approach on its own for food-demand prediction [3], while Hübner et al. went a step further and connected demand forecasting directly to environmental outcomes, showing in a bakery case study that better forecasts reduce the life-cycle impact of over-production [4]. Posch et al. took a Bayesian route instead of point forecasts, which is useful precisely when a canteen's sales history is too short for a confident regression estimate [5], and Anil Kumar et al. combined gradient-boosted ensembles (XGBoost and CatBoost together) with statistical features tied to location and customer satisfaction [6]. Taken as a group, these six studies confirm that demand forecasting for food service is a mature and well-studied problem, but none of them distinguish between institution types the way a corporate cafeteria, a college, and a school actually differ – which is exactly the distinction our forecasting engine is built around.

B. Personalized Food and Menu Recommendation

A separate line of work asks a different question: not how much food will be sold in aggregate, but what a specific person is likely to want. Ajami and Teimourpour built a hybrid recommender for an academic dining hall that combined collaborative filtering, content-based matching, and knowledge rules, and found an AdaBoost-based hybrid outperformed simpler baselines on a dataset of over 2,500 students [7]. MenuAI showed that transformer architectures developed for language tasks transfer reasonably well to restaurant menu recommendation [8], and Rokon et al. took the problem down to the ingredient level, pairing a CNN-based ingredient detector with a recipe-matching algorithm at 94% classification accuracy [9]. These systems are strong at predicting individual taste, but they say little about how a menu or a price should shift with the institutional segment being served, which is the gap our menu-recommendation and pricing engines are meant to fill rather than duplicate.

C. Reducing Waste Before It Happens

A third cluster targets waste directly, by changing what gets ordered, cooked, or offered before any of it becomes surplus. Aziz et al. framed menu selection itself as a fair-allocation problem, choosing a menu across people with different dietary needs and appetite sizes so as to minimize wastage under both optimistic and pessimistic assumptions about how much people actually eat [10]. Nu et al. measured, empirically, how much predictive and inventory-aware intervention actually reduces discarded food in restaurant operations [11], and both Ozcil [12] and Yang et al. [13] looked specifically at how well different ML models predict and reduce food waste. A broader survey of AI across the food-production-to-consumption pipeline situates this cluster within the larger landscape, and is one of the few sources to look at subsidized university dining specifically rather than commercial restaurants [14]. Unlike the donation literature discussed next, this cluster is about stopping waste on the supply side – which is exactly what our waste-prediction engine is built to inform, by feeding its leftover estimate back into both the kitchen's preparation quantity and the pricing markdown.

D. Distributing Surplus Food Fairly

The largest and most methodologically diverse cluster in our survey addresses what happens once food does become surplus. Dalal modeled food-donation planning under COVID-era supply and demand uncertainty as a robust optimization problem [15], while Sundin et al. assessed the sustainability of the donation pathway itself, treating it as a transfer system with measurable environmental, economic, and social value [16]. Maske et al. described the practical, technology-platform

side of connecting institutions with surplus to the organizations that can use it [17], and Sanyal et al. modeled donor-recipient interaction as a game rather than a one-sided optimization [18]. Two ScienceDirect studies formalize the allocation problem more rigorously: one balances effectiveness, efficiency, and equity through robust optimization and dynamic programming validated on real food-bank data [19], and another integrates demand forecasting directly into a multi-warehouse allocation model [20]. A 2025 study extends the same idea into vehicle routing, jointly optimizing delivery efficiency and fairness among partner agencies [21]. This is a genuinely deep literature, but nearly all of it is built for city-scale food banks serving many, often anonymous, recipients – not for an institutional cafeteria distributing surplus among a small, known, capacity-constrained set of NGO partners, which is the smaller and more relationship-aware version our NGO allocation engine actually solves.

E. Where This Leaves Us

Put together, no single cluster above covers more than two of the four capabilities this system needs, and, as our earlier survey concluded, none of the 21 reviewed sources addresses segment-aware pricing, targeted promotional conversion, an enforced cost-recovery floor, or coordination across multiple decision engines at once [22]. That survey proposed an architecture to close this gap; this paper is the report of what happened when we actually built and measured it, including the parts that needed correction once real numbers were in front of us.

III. System Architecture

The system runs as a single FastAPI service on top of a SQLAlchemy ORM over SQLite, chosen for development speed with the explicit understanding that swapping in PostgreSQL later is a one-line connection-string change rather than a rewrite. It exposes 40 REST endpoints to a React 19 / Vite / Tailwind CSS frontend, and authenticates every request with a JWT bearer token scoped to one of five roles: Super Admin, Kitchen Manager, Institution Coordinator, NGO Partner, and End Customer. Table I lists the stack as it is actually pinned in the project's dependency files, not as originally planned.

Rather than solving every decision with a single learned model, the system's seven engines deliberately use different methods, summarized in Table II. The reasoning is practical: pricing needs a hard, human-auditable floor that a kitchen manager must be able to trust and approve every day; NGO allocation needs constraints (dietary compatibility, partner capacity, a fairness guarantee) that must hold exactly, not just on average; and menu recommendation needs a justification a manager can read and act on, with no historical "correct menu" label available to train a ranking model against in the first place. Learned models are used only where the target genuinely benefits from statistical generalization – demand and leftover quantity, which are noisy continuous values with real day-to-day variation, and conversion, which is a clean, observable binary outcome.

TABLE I
Technology Stack as Implemented

Layer	Technology
Backend	FastAPI + Uvicorn, 40 REST endpoints
Database	SQLAlchemy ORM over SQLite, 19 tables
Authentication	JWT (python-jose) with pbkdf2 password hashing and role-based access control
ML libraries	XGBoost, scikit-learn (Random Forest, Gradient Boosting), pandas, numpy, joblib
Optimization	Google OR-Tools CP-SAT
Model storage	joblib model files plus a JSON metrics sidecar per trained model
Orchestration	A hand-written supervisor and seven-agent pipeline, each step audit-logged
Optional LLM	Ollama (local llama3.2), used only to phrase a briefing sentence; falls back to a fixed template; makes no decisions
External data	Open-Meteo live weather API, with an offline synthetic fallback
Frontend	React 19, Vite, Tailwind CSS, react-router-dom, Recharts
Testing	A 48-check end-to-end smoke test run against a live server

TABLE II
The Seven Engines and What Drives Each One

Engine	Method	Category
Demand Forecasting	XGBoost regression	Trained model
Dynamic Pricing + Floor	Closed-form elasticity formula	Deterministic
Waste / Surplus Prediction	Random Forest regression	Trained model
Menu Recommendation	Weighted multi-factor score	Rule-based
Trial-Conversion Targeting	Gradient Boosting classifier	Trained model
NGO Surplus Allocation	CP-SAT constraint solver	Exact optimization

These six engines are tied together by a seventh piece: a daily orchestration pipeline, run as a supervisor stepping through seven specialized agents rather than a distributed microservice setup (Figure 1). A Context agent starts by pulling live weather and calendar data; a Forecast agent predicts demand and a newsvendor-adjusted preparation quantity; a Waste agent uses that preparation quantity to predict how much will be left over; a Pricing agent turns the waste estimate and the demand forecast into a recommended price, always subject to the cost-plus-20% floor; a Menu agent ranks tomorrow's candidate dishes; an Offer agent scores yesterday's non-buyers for how likely a discount is to convert them; and an NGO agent turns the Waste agent's leftover prediction into surplus records and solves the fairness-constrained allocation problem across partner NGOs. A Supervisor closes the loop by assembling a summary, optionally phrasing it through a local LLM, and writing one audit row per agent – what it did, how many records it touched, how long it took, and whether it succeeded – to a persistent log. Each agent step is wrapped so that its own failure is caught and logged as an error rather than aborting the whole run; a bad forecast one day should not take down

pricing, menu, and NGO allocation with it. The full run, all seven agents, completes in about three seconds.

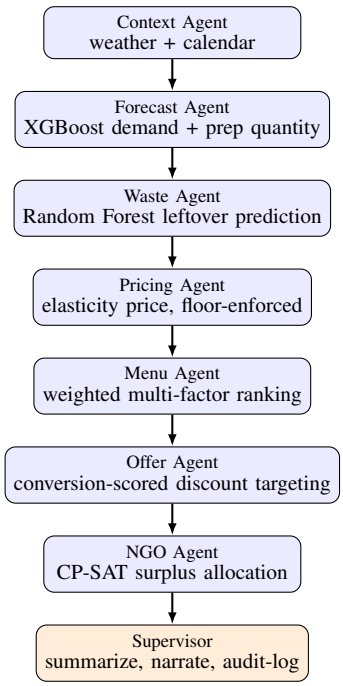


Fig. 1. The daily pipeline. Output from each stage feeds the next; a failure in any one agent is caught and logged rather than stopping the run.

IV. Dataset Generation

We looked for an existing public dataset before building our own and did not find one that fits. Datasets like UCI's Online Retail describe UK gift-ware e-commerce transactions, not food service, and none of the food-service datasets we found record unit cost, quantity prepared, quantity leftover, and institution segment together in one place – exactly the fields the pricing-floor, waste, and NGO engines all depend on. Following the approach our earlier survey identified as the accepted response to this gap [22], we built a synthetic generator instead, and fixed every generative parameter before training any model, so the reported results measure whether the models recover a documented structure rather than whether the structure was tuned after the fact.

Daily demand for dish d at institution i on day t is generated as

$$D(d,i,t) = B(i,t) \cdot S(d) \cdot W_{\text{dow}}(t,\sigma) \cdot W_{\text{wx}}(d,t) \cdot W_{\text{season}}(d,t) \cdot W_{\text{exam}}(d,i,t) \cdot T(d,t) \cdot P_e(d,\sigma) \cdot \epsilon \quad (1)$$

where $B(i,t)$ is the institution's base footfall that day, $S(d)$ is the dish's baseline share of that footfall, W_{dow} is a segment-specific day-of-week attendance curve, W_{wx} captures how weather-sensitive the dish is, W_{season} and W_{exam} account for seasonal and exam-period effects, $T(d,t)$ is a slow per-dish popularity drift, P_e applies the segment's own price elasticity, and ϵ is multiplicative noise.

Three institutions were modeled, each with a genuinely different profile: a corporate tech-park cafeteria (1,400 people, 62% daily turnout, busiest Tuesday through Thursday, sharp drop on Friday), a college (2,600 people, 48% turnout, steady across weekdays with a lighter Saturday), and a school (1,200 people, 71% turnout, uniform attendance with a half-day Saturday); all three close on Sunday. A 34-dish South Indian menu spans breakfast, mains, snacks, beverages, and desserts, and each dish carries a realistic unit cost, shelf life, spice and sweetness level, a vegetarian flag, and a weather-sensitivity coefficient – a buttermilk-style drink, for instance, sells more on hot days, while filter coffee sells more on cool ones. Menus differ by segment: schools drop non-vegetarian and caffeinated items, colleges drop the one premium dish, and the corporate cafeteria runs the full menu at a higher cost basis with the steepest markup of the three. Weather itself is generated from a seasonal temperature curve with daily noise and a monsoon rainfall model, giving the forecaster real structure to find rather than pure randomness. Calendar effects include an actual 2025–26 holiday list and college exam-period windows, which suppress full-meal demand while pushing up snacks and beverages. Price elasticity differs sharply by segment, from -0.6 for the price-tolerant corporate segment to -2.1 for the highly price-sensitive school segment. Kitchens in the simulation deliberately over-prepare by a shelf-life-dependent margin to avoid running out, which is precisely what creates the leftover signal the waste engine is trained to predict.

The generator produces 365 days of aggregate sales across the three institutions, 29,044 rows in total, plus 45 days of individual, per-customer orders layered on top of that. Around 500 synthetic customers each carry a persistent taste bias, three to six favorite dishes and a loyalty score, so that the trial-conversion engine has real repeat-purchase patterns to learn from rather than uniformly random behavior. Feedback ratings are correlated with each dish's underlying popularity, again with noise added, so the menu engine's satisfaction term reflects something real. All random seeds are fixed, so the dataset is fully reproducible.

V. Methodology

A. Demand Forecasting

The forecasting engine predicts how many units of a given dish will sell at a given institution on a given day, using a single XGBoost regression model trained across every institution and dish together rather than a separate model per dish. With 34 dishes across three institutions, some with a full year of history and others recently added, pooling the data lets a data-poor dish borrow statistical strength from similar ones through shared features like category, spice level, and price, instead of requiring its own long time series. The model consumes 24 features spanning calendar signals, live weather and footfall, dish attributes, and lag/rolling statistics computed one day behind the target row so that no row can see its own future value.

We evaluated the model on the final 30 days of the dataset, held out entirely from training, since a random cross-validation split would let future information leak into the past and make the model look better than it is. Against a naive baseline that simply predicts each day's demand as the trailing 7-day average – roughly the heuristic a busy kitchen manager already uses – the model cuts mean absolute error by 45.0%, reaching an R^2 of 0.875 and a MAPE of 19.2% on the holdout. To turn a point forecast into an actual quantity to cook, the system adds a safety margin sized from the model's own recent forecast error for that dish, rather than from raw historical demand variance. Sizing the margin from raw variance double-counts variation the model has already explained and, in our own testing, pushed over-preparation up to roughly 44%; sizing it from forecast error instead brought that down to about 12%. Dishes with a short shelf life get a thinner margin than long-lasting ones, since a stockout there is a smaller loss than the waste from over-preparing something that spoils in a few hours.

B. Dynamic Pricing and the Cost-Recovery Floor

Pricing is the one decision a human kitchen manager approves every single day, and it comes with a constraint that must never be broken: the selling price can never fall below cost. For that reason, price comes from a closed-form economic model rather than a learned one. Starting from a constant-elasticity profit function,

$$\pi(p) = (p - c)\, D_0 \left(\frac{p}{p_0}\right)^{e}, \tag{2}$$

the profit-maximizing price works out to

$$p^{*} = \frac{c \cdot e}{e + 1}, \qquad \text{valid only when } e < -1, \tag{3}$$

where c is unit cost, D_0 is baseline demand at reference price p_0, and e is the segment's price elasticity. This formula breaks down for the corporate segment, whose elasticity of -0.6 is inelastic – profit keeps rising as price rises, with no interior maximum – so the engine falls back to a fixed segment ceiling instead of extrapolating an unbounded price. That ceiling is also, conveniently, the mechanism that answers a fairness objection raised early in the project: differentiated pricing by segment is bounded, not open-ended. From there, the unconstrained target price is nudged up or down by up to 8% depending on current demand pressure, marked down by up to 18% when the waste engine flags a high leftover risk, clamped to the segment's allowed price band, and finally passed through an unconditional floor,

$$p_{\text{final}} = \max\big(p_{\text{adjusted}},\ 1.20\, c\big), \tag{4}$$

applied last so that nothing upstream of it – not a markdown, not an elasticity estimate, not a promotional discount – can push the price below cost recovery. The API also rejects a manual override from a manager that would violate this floor.

C. Waste and Surplus Prediction

This engine predicts how many units of a dish will be left unsold at the end of service, using the kitchen's own decision about how much to prepare as an input feature. That is a legitimate feature rather than a leak, since the kitchen commits to a preparation quantity before demand for the day is actually observed. We treat this as a regression problem rather than classification, because the pricing markdown and the NGO allocator both need an actual number of units, not just a high/medium/low label; the label shown on the kitchen's dashboard is derived from the predicted quantity afterward. We picked Random Forest over a boosted model here specifically because leftover counts are noisier and more skewed toward zero than raw demand – mostly small integers with occasional spikes when a kitchen clearly over-prepares – and averaging across 250 fully grown trees handles that kind of noise more gracefully than a single boosted sequence, without needing the careful tuning a gradient booster would require to avoid overfitting sparse counts. Evaluated on the same 30-day holdout used for forecasting, the model reaches an R^2 of 0.839, and its predicted risk band (high, medium, or low) matches the actual outcome closely enough to be useful on the kitchen's own traffic-light display.

D. Menu Recommendation

There is no historical record of an "ideal" menu to train a ranking model against, so this engine instead computes a transparent score for each candidate dish:

$$\begin{aligned}\text{score}(d) = w_{\text{pop}}\cdot\text{pop}(d) + w_{\text{trend}}\cdot\text{trend}(d) + w_{\text{sat}}\cdot\text{rating}(d) \\ + w_{\text{profit}}\cdot\text{margin}(d) + w_{\text{fresh}}\cdot(1 - \text{repetition}(d)) \\ + w_{\text{ctx}}\cdot\text{context}(d),\end{aligned} \tag{5}$$

where trend is measured as a 7-day average relative to a 28-day average, so the engine reacts to momentum rather than raw popularity, and the weight vector changes by segment: colleges weight trend most heavily, matching a population that chases whatever is new; the corporate segment weights satisfaction and margin most; schools weight popularity and menu variety most. Each recommended dish comes back with a short, human-readable reason built from the same numbers, such as noting that a dish is trending up or hasn't been served recently, which gives a kitchen manager something to actually evaluate rather than a bare ranked list.

E. Trial-Conversion Targeting

This engine looks at yesterday's best-selling dish, isolates the customers who visited but did not buy it, and scores each of them for how likely a discount is to change their mind, using a Gradient Boosting classifier trained on eleven behavioral features – recent visit frequency, category affinity, whether they have ever bought that dish, days since their last visit, and similar signals. Customers who never visit at all are excluded, since a discount can't reach them, and customers who already buy the dish are excluded too, since discounting them just

gives away margin for nothing. The training labels come from walking 45 days of order history and checking, for each non-buyer/dish/day combination, whether that customer actually bought the dish within the following week – a real behavioral outcome, not a synthetic proxy. We chose Gradient Boosting over logistic regression because these features interact in ways that are hard to write down by hand (whether someone has ever bought a specific dish matters very differently depending on how often they visit at all), and shallow boosted trees pick that up without manual feature-crossing. The model reaches an AUC of 0.658, and, more relevant to the actual business decision, customers in the top-scored 20% convert at 2.03× the base rate – meaning a fixed discount budget spent on the highest-scored customers reaches roughly twice as many actual conversions as spreading the same budget across everyone. We are careful to describe this as a targeting-quality result rather than a causal one, since the labels come from observed behavior rather than a randomized experiment.

F. NGO Surplus Allocation

There is no historical record of a "correct" surplus split either, and unlike the menu engine's soft preferences, this decision has constraints that genuinely must hold: surplus can't be allocated beyond what exists, vegetarian-only NGOs can't receive non-vegetarian food, no partner can be given more than its stated capacity, and every NGO selected for a given day must receive at least 20% of that day's allocatable surplus. A learned model could only be hoped to respect constraints like these; a constraint solver enforces them exactly, which is why this engine is built on Google OR-Tools' CP-SAT solver rather than any form of machine learning. The variables are how many units of each surplus item go to each NGO and whether each NGO participates at all that day, subject to linear constraints for each rule above, with an objective that first maximizes total food distributed, then need-weighted fairness, then worst-case coverage across partners.

One issue surfaced only once we started testing with real numbers: with five candidate NGOs, a flat 20% guarantee for all of them adds up to exactly 100%, leaving the solver nothing to actually optimize. We fixed this by applying the guarantee only to the NGOs selected for that day's run rather than all candidates. A second, subtler problem followed from the first: capping the guarantee purely by percentage meant that on a high-surplus day, 20% of the total could exceed what a small NGO can physically take, which disqualified that partner and reduced overall coverage – exactly the opposite of what the guarantee was meant to do. Capping the 20% guarantee by each partner's own stated capacity fixed this, and raised measured surplus coverage from roughly 76% to 99.4% in our testing, while still keeping every selected partner above its guaranteed share on every run.

G. Tying It Together

The six engines above run in the fixed order shown in Figure 1, with each stage's output available to the ones after it; the forecast feeds the waste prediction, and the waste prediction feeds both the pricing markdown and the NGO surplus records. The three trained models are trained offline, ahead of time, each producing a saved model file and a metrics record; the daily pipeline only ever loads and predicts with whatever model is currently saved, which keeps the live run fast and keeps every day's decisions traceable back to a specific model version. Every agent's action, on every run, is written to a persistent audit log recording what it did, how many records it touched, how long it took, and whether it succeeded – something a simple, non-agentic script would not have given us for free.

VI. Results

Table III summarizes what we measured, drawn from our 48-check automated test suite and the system's own internal reporting endpoint, run against the generated dataset described in Section IV.

TABLE III
Measured Results Against the Generated Dataset

Metric	Value
Forecast R^2 (30-day holdout)	0.875
Forecast MAPE	19.2%
Forecast MAE improvement over naive baseline	45.0%
Waste model R^2	0.839
Trial-conversion AUC	0.658
Lift over blanket discounting (top 20% targeting)	2.03×
Recommended over-preparation (vs. 14.2% baseline)	12.2–12.7%
Surplus successfully redistributed	99.4%
NGO 20% guarantee met	every selected partner, every run
Price-floor violations observed	0
Full daily pipeline run time	≈3 s

Figures 2 and 3 show the live system producing these numbers rather than a mock-up. Figure 2 is the Kitchen Manager's Today's Dashboard immediately after a daily pipeline run: all eight agents completed in 6.35 seconds and the summary line reports a concrete, auditable example of the pipeline's output for that one day – a demand forecast of 2,979 units across 34 dishes at the corporate institution, a recommended preparation quantity of 3,374 units, 34 price adjustments worth Rs. 79,680 in additional margin with zero dishes held at the cost-recovery floor, 10 non-buyers targeted for trial conversion, and 98 of 98 surplus units routed to 4 NGO partners (100% coverage that day, guarantee satisfied). Figure 3 is the corresponding Demand Forecast page, showing the XGBoost model's held-out R^2 of 0.8752 and MAPE of 19.21%, a 44.99% improvement over the naive rolling-mean baseline on 2,417 held-out test rows, and the seven-day institution-level demand outlook the kitchen actually plans against.

The forecasting model's 45.0% improvement over a naive rolling average suggests it is capturing something beyond what a kitchen manager could estimate by eye, and sizing the preparation-quantity safety margin from the model's own error, rather than from raw demand variance, kept over-preparation

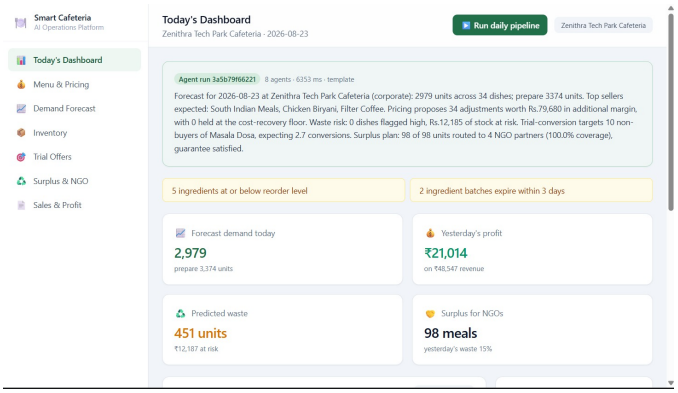


Fig. 2. Kitchen Manager's Today's Dashboard immediately after a live daily pipeline run, showing the agent-run summary and the resulting forecast, profit, waste, and NGO surplus figures for one day at the corporate institution.

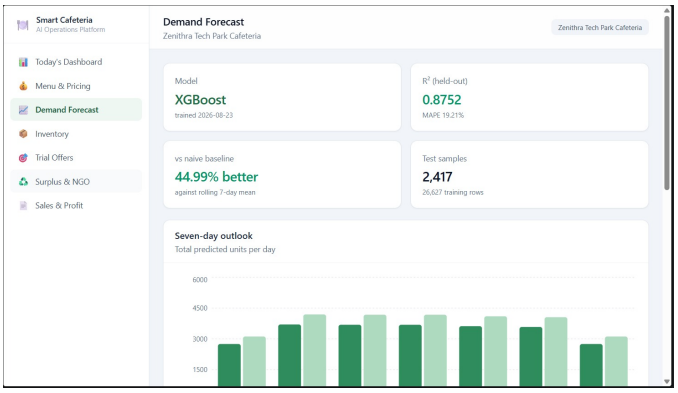


Fig. 3. Demand Forecast page reporting the XGBoost model's held-out evaluation metrics and the seven-day demand outlook generated from the live model.

close to the naive baseline instead of the roughly 44% we saw when we first tried sizing it the naive way. The trial-conversion engine's 2.03× lift indicates that a discount budget spent on the highest-scored customers does meaningfully better than spreading it evenly, though we want to repeat that this is a targeting-quality number rather than proof of a causal effect, since we did not run a randomized holdout group. The NGO allocation fix described in Section V-F took measured surplus coverage from about 76% to 99.4% while still satisfying the 20% guarantee for every partner on every run, and across every pricing decision the system made during testing, the cost-recovery floor was never once breached, which was the property it was specifically designed to guarantee.

VII. Discussion

The choice to use six genuinely different methods across six engines, rather than picking one approach and applying it everywhere, was deliberate rather than a shortcut. A learned model cannot promise a hard price floor or an exact fairness guarantee, and a menu recommender with no ground truth to train against gains more from being explainable than from being a black box. Machine learning earns its place only in the three spots where the target genuinely benefits from statistical generalization over noisy real data: demand, leftover quantity, and conversion probability.

We want to be direct about this project's limitations rather than let an examiner find them first. The dataset behind every number in Section VI is synthetic; what these results demonstrate is that the pipeline recovers a documented, inspectable structure, not that these exact figures will reproduce on a live cafeteria's real data. The database layer is SQLite rather than PostgreSQL, which was a timeline decision rather than a technical one, and moving to PostgreSQL later is a connection-string change rather than a redesign. The optional LLM narration step runs on a fixed template in most environments rather than an actual local model, though this only affects the wording of a summary sentence and touches no decision logic. As our own earlier survey pointed out, the pricing and promotion engines rest on a thinner base of directly relevant literature than the forecasting, waste, and NGO-distribution engines do, each of which maps onto a specific, well-populated cluster of prior work [22]; closing that gap with literature specifically on segment-based retail pricing and multi-agent coordination is future work rather than something this paper claims to have solved. Finally, the 2.03× conversion lift, as noted above, measures targeting quality on observed behavior, not a causal treatment effect, and should be read that way.

VIII. Conclusion and Future Work

We set out to build, not just propose, a system that could forecast demand, price food fairly across different institutional segments, recommend menus with a real justification attached, and route surplus food to NGO partners equitably, closing a gap our own earlier literature survey had identified but not filled [22]. What we ended up with is seven engines – three trained models, one exact optimizer, one rule-based scorer, and one closed-form pricing rule – run daily by a supervised, fault-isolated pipeline that completes in about three seconds and logs every decision it makes. Measured against a documented synthetic dataset with a strict time-based holdout, the system reaches a demand-forecasting R^2 of 0.875, a waste-prediction R^2 of 0.839, a conversion AUC of 0.658 with 2.03× targeting lift, 99.4% surplus coverage under a guaranteed per-partner fairness rule, and zero observed breaches of its cost-recovery floor. Along the way we found and fixed a specific fairness/capacity interaction in the NGO allocator, which we consider one of the more concrete, defensible findings in this work.

There is a reasonably clear list of what comes next. The most important step is validating the pipeline against real data from an actual cafeteria rather than synthetic data alone, ideally through a pilot at one of our own institutions. Billing and payment-gateway integration, explicitly out of scope for this phase, would need to be added before the system could run unattended in production. A randomized or quasi-experimental test of the trial-conversion engine would let us claim a causal effect rather than a targeting-quality proxy. The pricing and promotion engines would benefit from a dedicated literature pass on dynamic retail pricing and multi-agent coordination,

an evidentiary gap our own survey flagged and this paper has not closed. And the NGO allocation engine currently depends on partners self-reporting their needs; predicting need instead of asking for it directly is a natural next step once the core system has been validated in the field.

ACKNOWLEDGMENT

We thank our project guide, Ms. M. Bhavani, Assistant Professor, Department of Computer Science and Engineering, Rajalakshmi Engineering College, for her guidance through both the survey and implementation phases of this project.

REFERENCES

[1] B. Aydın, Y. S. Balcıoğlu, and B. Sezen, "Machine Learning Techniques for Cafeteria Demand Forecasting: An Institutional Case," *OPUS J. Society Research*, 2025.

[2] M. Aci and D. Yergök, "Demand Forecasting for Food Production Using Machine Learning Algorithms: A Case Study of University Refectory," 2023.

[3] M. Agarwal, S. Kulkarni, V. Nagre, A. Joshi, and D. Nagpure, "Food Demand Forecasting Using Machine Learning and Statistical Analysis."

[4] N. Hübner, J. Caspers, V. C. Coroama, and M. Finkbeiner, "Machine-Learning-Based Demand Forecasting Against Food Waste: Life Cycle Environmental Impacts and Benefits of a Bakery Case Study," 2024.

[5] K. Posch, C. Truden, P. Hungerländer, and J. Pilz, "A Bayesian Approach for Predicting Food and Beverage Sales in Staff Canteens and Restaurants," 2022.

[6] D. Anil Kumar, B. Thirupathi Rao, B. Rangaswamy, and K. Meghana, "An Efficient Approach for Food Demand Forecasting Using an Ensemble Technique and Statistical Analysis," in *Proc. Int. Conf. Cognitive Computing and Cyber Physical Systems (IC4S)*, Springer, 2025, doi: 10.1007/978-3-031-77075-3_6.

[7] A. Ajami and B. Teimourpour, "A Food Recommender System in Academic Environments Based on Machine Learning Models," arXiv:2306.16528, 2023.

[8] X. Ju, F. P. W. Lo, J. Qiu, P. Shi, J. Peng, and B. Lo, "MenuAI: Restaurant Food Recommendation System via a Transformer-Based Deep Learning Model."

[9] Md. S. J. Rokon, Md. K. Morol, I. B. Hasan, A. M. Saif, and R. H. Khan, "Food Recipe Recommendation Based on Ingredients Detection Using Deep Learning," in *Proc. 2nd Int. Conf. Computing Advancements (ICCA)*, ACM, 2022, doi: 10.1145/3542954.3542983.

[10] H. Aziz, N. Mattei, S. Narang, and S. Roy, "Menu Selection: A Computational Approach to Minimizing Food Waste," arXiv:2606.06989, 2026.

[11] Y. Nu, E. Belavina, and K. Girotra, "Using Artificial Intelligence to Reduce Food Waste," 2024.

[12] I. E. Ozcil, "Reducing Food Waste Using Machine Learning Models: Forecasting and Optimization Approaches," 2024.

[13] Y. Yang, C. Zhao, and H. Lu, "Leveraging Machine Learning for Food Waste Reduction: An Analysis of Predictive Models," 2024.

[14] "From Plate to Production: Artificial Intelligence in Modern Consumer-Driven Food Systems," arXiv:2311.02400, 2023.

[15] J. Dalal, "Food Donation Management Under Supply and Demand Uncertainties in COVID-19: A Robust Optimization Approach," 2022.

[16] N. Sundin, L. Bartek, C. Persson Osowski, I. Strid, and M. Eriksson, "Sustainability Assessment of Surplus Food Donation: A Transfer System Generating Environmental, Economic, and Social Values," 2023.

[17] R. Maske, R. Wagh, A. Verma, O. Thopate, and B. Bhagat, "A View on Surplus Food Donation App."

[18] S. Sanyal, V. K. Singh, F. Xhafa, B. Sanyal, and S. Mukhopadhyay, "A Game Theoretic Framework for Surplus Food Distribution in Smart Cities and Beyond."

[19] "Robust Optimization Approaches for the Equitable and Effective Distribution of Donated Food," ScienceDirect, 2018.

[20] "Predicting and Optimizing the Fair Allocation of Donations in Hunger Relief Supply Chains," ScienceDirect, 2024.

[21] "Collaborative Vehicle Routing for Equitable and Effective Food Allocation in Nonprofit Settings," *J. Humanitarian Logistics and Supply Chain Management*, Emerald, 2025.

[22] Kamalesh P, Mithul Aaditya A, Jayaprakash A, and Karthika Lakshmi V, "A Survey of AI-Driven Demand Forecasting, Personalized Food Recommendation, Waste-Prevention Optimization, and Equitable Surplus Redistribution in Institutional Cafeteria Management," Dept. of Computer Science and Engineering, Rajalakshmi Engineering College, project technical report, 2026.

[23] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[24] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[25] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[26] L. Perron and V. Furnon, "OR-Tools," Google, [Online]. Available: https://developers.google.com/optimization.